

ARCHITECTURE TECHNIQUE DÉTAILLÉE

Plateforme Finance & Trésorerie — SIMTIS

Next.js • FastAPI • PostgreSQL • Docker

Couche	Technologie / rôle
Frontend	Next.js — interface utilisateur et écrans métier.
API / Backend	FastAPI — API REST, sécurité et logique métier.
Services métier	Import, normalisation, position, rapprochement, prévisions, audit.
Base de données	PostgreSQL — stockage des données normalisées et historiques.
Déploiement	Docker / Docker Compose.
Sources	Banques + Sage / SI + fichiers Excel/CSV selon les données disponibles.

Principe fondamental : la plateforme est une couche d'intégration, de contrôle et de pilotage. Elle ne remplace pas Sage / SI et ne recrée pas les opérations comptables déjà réalisées.

Architecture technique basée sur le document technique SIMTIS fourni et détaillée pour servir de référence au développement.

1. OBJECTIF DE L'ARCHITECTURE TECHNIQUE

Construire une application modulaire qui récupère les données des banques et les résultats déjà produits par Sage / SI, les normalise, automatise la position bancaire et le rapprochement, puis expose les résultats aux utilisateurs.

1.1 Architecture globale

UTILISATEUR → NEXT.JS → HTTPS / REST API → FASTAPI → SERVICES MÉTIER → POSTGRESQL

BANQUES + SAGE / SI → IMPORT → NORMALISATION → POSTGRESQL → TRAITEMENTS → NEXT.JS

Couche	Responsabilité
Utilisateur	Consulter, importer, valider, contrôler et piloter.
Next.js	Interface, navigation, formulaires, tableaux, KPI.
FastAPI	API REST, authentification, autorisation et orchestration métier.
Services	Logique métier spécialisée.
PostgreSQL	Persistance et relations entre données.
Banques / Sage	Sources externes.

2. FRONTEND — NEXT.JS

Le frontend est l'interface utilisée par la Trésorerie, la Comptabilité, les responsables et la Direction. Il ne doit pas communiquer directement avec PostgreSQL : toutes les données passent par FastAPI.

2.1 Modules frontend

Route / module	Fonction
/dashboard	KPI, position, rapprochement, écarts, prévisions, alertes.
/banques	Liste des banques et comptes.
/banques/[id]	Détail d'une banque / compte.
/relevés	Import, historique et consultation des relevés.
/position-bancaire	Soldes, crédits et position disponible.
/rapprochement	Transactions bancaires, écritures et propositions.
/ecarts	Suivi des opérations non rapprochées et écarts.
/previsions	Prévisions et position future.
/historique	Historique des opérations et validations.
/administration	Utilisateurs, rôles et permissions.

2.2 Structure recommandée

frontend/ └─ app/ | └─ login/ | └─ dashboard/ | └─ banques/ | └─ relevés/ | └─ position-bancaire/ | └─ rapprochement/ | └─ ecarts/ | └─ previsions/ | └─ administration/ | └─ composants/ | └─ services/ | └─ hooks/ | └─ types/ | └─ lib/ | └─ public/

Rôle des dossiers

Dossier	Rôle
app/	Pages et routes Next.js.
composants/	Composants réutilisables.
services/	Appels HTTP vers FastAPI.
hooks/	Gestion des données et états React.
types/	Types TypeScript.
lib/	Client API, authentification et utilitaires.
public/	Ressources statiques.

3. BACKEND — FASTAPI

FastAPI constitue la couche API et la logique métier. Le frontend envoie des requêtes HTTPS/REST et le backend contrôle les droits, exécute le traitement puis accède à PostgreSQL.

3.1 Architecture backend

backend/ └─ app/ └─ api/ | └─ banks.py | └─ accounts.py | └─ statements.py | └─ reconciliation.py | └─ forecasts.py | └─ dashboard.py └─ services/ | └─ bank_service.py | └─ import_service.py | └─ normalization_service.py | └─ position_service.py | └─ reconciliation_service.py | └─ forecast_service.py | └─ audit_service.py └─ models/ └─ schemas/ └─ repositories/ └─ main.py

3.2 Rôle de chaque couche

Couche	Responsabilité
api/	Endpoints HTTP ; reçoit les requêtes et renvoie les réponses.
services/	Contient les règles métier.
models/	Modèles ORM représentant les tables PostgreSQL.
schemas/	Validation des données entrantes/sortantes.
repositories/	Accès à PostgreSQL.
main.py	Démarrage FastAPI et configuration principale.
app/	Configuration, sécurité, base de données et dépendances.

4. SERVICES MÉTIER

Service	Responsabilité
Bank Service	Banques, comptes, soldes, crédits et informations bancaires.
Import Service	Réception et traitement des fichiers bancaires/comptables.
Normalization Service	Mapping et transformation vers les modèles standards.
Position Service	Calcul du crédit disponible et de la position disponible.
Reconciliation Service	Matching banque ↔ comptabilité : 1→1, 1→N, N→1, N→N.
Forecast Service	Réalisé, prévu, en attente, reporté et position future.
Audit Service	Historisation des actions importantes.

Principe

Les API ne doivent pas contenir toute la logique métier. Elles appellent les services correspondants afin de garder une architecture modulaire et maintenable.

5. IMPORT DES DONNÉES BANCAIRES

Excel / CSV / MT940* → Import Service → Détection format → Mapping → Validation → Normalisation → PostgreSQL

5.1 Flux détaillé

Étape	Traitement
1. Upload	Next.js envoie le fichier à FastAPI.
2. Détection	Le backend identifie les colonnes et le format.
3. Mapping	Association entre colonnes bancaires et champs standard.
4. Validation	Contrôle des dates, montants, banque, compte et erreurs.
5. Normalisation	Transformation vers le modèle interne.
6. Stockage	Enregistrement dans bank_statements et bank_transactions.
7. Résultat	Retour au frontend : lignes importées, erreurs, doublons.

5.2 Structure standard

La structure métier comprend notamment : Société, Pointage, Banque, Date d'opération, Date de valeur, Libellé, Débit, Crédit, Solde, Lettrage/Escompte et Commentaire.

6. IMPORT DES DONNÉES SAGE / SI

Sage / SI → Export / interface / API* → Accounting Import Service → Validation → Normalisation → accounting_entries

Les écritures ne sont pas recréées dans la plateforme. Elles sont récupérées depuis le système source afin d'alimenter le rapprochement.

6.1 Structure comptable utilisée pour le rapprochement

Champ	Contenu
Date	Date de l'écriture.
Libellé	Description comptable.
Débit	Montant au débit.
Crédit	Montant au crédit.
N° pièce	Référence de la pièce.
Échéance	Échéance associée lorsqu'elle existe.

6.2 Contrôle

L'import doit signaler les lignes invalides, les doublons potentiels et les champs obligatoires manquants avant intégration.

7. SERVICE DE NORMALISATION

Tous les relevés et écritures doivent être convertis vers un modèle interne commun afin de permettre les traitements indépendamment du format de la source.

Donnée standard	Exemples de traitement
Date opération	Conversion vers un format de date unique.
Date valeur	Conversion et contrôle.
Date comptable	Date issue de la source comptable.
Libellé	Nettoyage et conservation du texte source.
Référence	Extraction lorsque disponible.
Débit / Crédit	Normalisation des signes et formats numériques.
Montant	Format numérique standard.
Solde	Conservation ou recalcul selon la source.
Compte	Association au compte bancaire.
Source	Banque, Sage/SI ou fichier.
La normalisation permet ensuite aux services Position et Rapprochement de travailler sur des données homogènes.	

8. SERVICE DE RAPPROCHEMENT

Transactions bancaires + Écritures comptables → Moteur de matching → Proposition → Validation → Statut

8.1 Critères

Montant • Date • Référence • Libellé • Sens débit/crédit • Numéro de chèque • N° pièce lorsque disponible

8.2 Types

Type	Traitement
1→1	Une transaction bancaire ↔ une écriture.
1→N	Une transaction bancaire ↔ plusieurs écritures.
N→1	Plusieurs transactions ↔ une écriture.
N→N	Plusieurs transactions ↔ plusieurs écritures.

8.3 Statuts

Rapprochée / À vérifier / Non rapprochée / Écart

Le moteur propose les correspondances. La validation finale reste soumise aux droits de l'utilisateur.

9. POSITION SERVICE

Le Position Service calcule les informations de position à partir des soldes bancaires et des crédits.

Solde bancaire + Crédit autorisé – Crédit utilisé → Crédit disponible → Position disponible

Indicateur	Formule
Crédit disponible	Crédit autorisé – Crédit utilisé
Position disponible	Solde bancaire + crédit disponible

Le calcul doit être disponible par compte, par banque et globalement.

9.1 Exemple

Solde bancaire = 300 000 MAD

Crédit autorisé = 500 000 MAD

Crédit utilisé = 100 000 MAD

Crédit disponible = 400 000 MAD

Position disponible = 700 000 MAD

10. FORECAST SERVICE

Transactions réalisées + opérations prévues + échéances → Forecast Service → Position prévisionnelle

10.1 Statuts

Statut	Signification
Réalisé	Flux déjà exécuté.
Prévu	Flux attendu.
En attente	Flux attendu mais non confirmé.
Reporté	Flux déplacé à une nouvelle date.

10.2 Catégories

Encaissement • Escompte • Douane • Paie • Refinancement • Chèques • Autres

10.3 Calcul

Position prévisionnelle = position de départ + encaissements prévus – décaissements prévus

11. POSTGRESQL — MODÈLE DE DONNÉES

Table	Rôle
banks	Référentiel des banques.
bank_accounts	Comptes bancaires.
bank_statements	Relevés importés.
bank_transactions	Mouvements bancaires normalisés.
accounting_entries	Écritures récupérées de Sage/SI.
reconciliation_matches	Associations entre banque et comptabilité.
reconciliation_rules	Règles de rapprochement.
discrepancies	Écarts et suivi.
forecasts	Prévisions de trésorerie.
users	Utilisateurs et rôles.
audit_logs	Historique des actions.

11.1 Relations

Bank 1→N Bank Accounts

Bank Account 1→N Bank Statements

Bank Statement 1→N Bank Transactions

Bank Transaction N↔N Accounting Entry via Reconciliation Match

12. SÉCURITÉ ET AUTORISATION

Utilisateur → Authentification → Session/JWT → Rôle → Permissions → API

Profil	Accès principal
Trésorerie	Banques, position, mouvements, prévisions.
Comptable	Rapprochement, écarts, contrôles.
Responsable	Validation et supervision.
Direction	Dashboard et reporting.
Administrateur	Utilisateurs, rôles et configuration.
Le backend doit vérifier les permissions avant les opérations sensibles : import, validation, rapprochement, modification et administration.	

13. DÉPLOIEMENT DOCKER

Le MVP peut être déployé avec Docker Compose.

docker-compose |— frontend → Next.js |— backend → FastAPI |— database → PostgreSQL

Flux

Navigateur → Frontend Next.js → FastAPI → PostgreSQL

Les fichiers importés sont transmis au backend qui effectue les contrôles et traitements avant leur enregistrement.

Évolution possible

Redis ou un worker asynchrone peut être ajouté plus tard si les imports ou traitements deviennent volumineux. Il n'est pas indispensable pour le MVP.

14. FLUX COMPLET DES DONNÉES

N°	Flux technique
1	Utilisateur ouvre Next.js.
2	Next.js appelle une API FastAPI.
3	FastAPI vérifie l'authentification et les permissions.
4	FastAPI appelle le service métier concerné.
5	Le service lit ou écrit PostgreSQL via les repositories.
6	Le service applique les règles métier.
7	FastAPI renvoie une réponse JSON.
8	Next.js actualise l'interface.

Exemple — import d'un relevé

Next.js Upload → POST /statements/import → Import Service → Normalization Service → Repository → PostgreSQL → réponse JSON → tableau Next.js.

Exemple — rapprochement

Next.js → POST /reconciliation/run → Reconciliation Service → bank_transactions + accounting_entries → matching → reconciliation_matches → réponse → validation utilisateur.

15. ARCHITECTURE TECHNIQUE FINALE

UTILISATEURS

↓

NEXT.JS

↓ HTTPS / REST

FASTAPI

|— Bank Service

|— Import Service

|— Normalization Service

|— Position Service

|— Reconciliation Service

|— Forecast Service

|— Audit Service

↓

POSTGRESQL

↑

BANQUES + SAGE / SI

15.1 MVP technique recommandé

Étape	Développement
1	Next.js + FastAPI + PostgreSQL + Docker.
2	Gestion des banques et comptes.
3	Import d'un premier format de relevé bancaire.
4	Normalisation des transactions.
5	Calcul de la position bancaire.
6	Import d'un export Sage/SI.
7	Moteur de rapprochement 1→1 puis 1→N / N→1.
8	Gestion des écarts et validation manuelle.
9	Dashboard et prévisions.

15.2 Règle d'architecture importante

L'application doit rester une couche d'intégration, de contrôle et de pilotage. Elle ne doit pas devenir un deuxième Sage. Les données existantes doivent être récupérées autant que possible afin d'éviter les doubles saisies et les incohérences.

Avant l'implémentation définitive, les formats bancaires, l'accès à Sage/SI et la disponibilité éventuelle d'API doivent être confirmés techniquement avec SIMTIS.

Fin — Architecture technique détaillée SIMTIS